

LINUX :-

Wsl → windows sub system for linux.

- Linux is an open source OS based on UNIX.
- Just like windows or macOS, it is a software that manages your computer's hardware and lets you run programs & applications.

LINUX is

- open source → anyone can view, modify & distribute the source code
- free to use
- secure
- customizable
- Huge community support.

Kernel :-

- * It is a core part of any operating system.
- * It acts as a bridge b/w software & hardware

Kernel is like a manager that:

- controls CPU, memory & devices (like keyboard, mouse)
- make sure each program gets the resource it needs.
- keeps everything running smoothly & securely.

Functions :-

- 1) Process management :- start, stops and switches b/w running programs.
- 2) memory management :- allocate RAM to programs & keep them separate.
- 3) Device management :- lets software talk to hardware using drivers
- 4) System calls / API :- provides an interface for apps to request services.
- 5) File system control :- manages files and directories on storage devices.

Types :

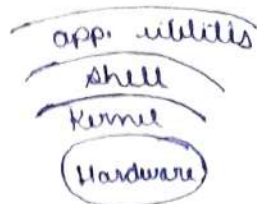
- 1) Monolithic Kernel :- all functions runs together, (linux)
- 2) Micro kernel :- only essential parts runs in the core, others in user space (minix)
- 3) Hybrid Kernel :- min of both (windows, macOS)

Diff. b/w Linux & Windows

Linux

- * open source, free of cost
(GPL → general public license)
- * uses monolithic kernel
- * file names are case sensitive.
file.txt & File.txt is diff. from each other
- * uses / (forward slash) for directory separation
- * more secure
- * 3 types of user acc.
Regular, Root, service acc.

Linux Architecture



Windows

- * paid and requires licenses.
- * uses hybrid kernel.
- * file names are case insensitive
- * uses \ (backslash) for directory separation.
- * less secure.
- * 4 types of user acc.
Admin, standard, child, guest.

vmstat → command that shows you the real-time performance of your system.

Lib: CPU usage, memory usage, Processes, I/O (disk read/write)

Shell:- a shell is a program that takes your commands and tells the computer what to do.

Types of shell:-

- 1) Command Line shell (CLI):- bash, zsh, sh. → eg.
in CLI we type commands (eg. ls, mkdir, cd)
- 2) Graphical shell (GUI):- GNOME | KDE
visual way to interact

e.g.- when we type this in terminal:

mkdir new-folder (You type command)

- The shell understands you want to make a new folder
- It tells kernel to do it.
- The folder appears in your file system.

most common Linux shell is

Bash = ~~Bourne~~ Bourne Again Shell

Shell :- Talks to user (interface to OS).

Kernel :- Talks to hardware (heart of OS)

boot

Bootloader:- a small program that runs when a computer starts.

- Its job is to load the operating system into memory so that system can start working.

Think of bootloader as starter of OS.

When you press power button of CPU

- Hardware starts;
- Bootloader runs.
- It loads OS
- OS takes over and shows home screen or login screen.

eg:-

- 1) GRUB (Grand Unified Bootloader)
 - 2) LILO
 - 3) U-Boot etc.
- Linux Loader

Commands :-

`cd ..` → parent directory
`cd .` → current directory.

1) `cd ..` → help us to move to prev folder.
go back to prev folder.

2) `rm -r` → delete a folder and everything inside it.
↓ recursive
removes (go inside folders & delete everything)

e.g. If you have folder called `project` and you run

`rm -r project`

It will delete

→ the `project` folder

→ all files inside it

→ any subfolders inside it.

`rm -r project/*`

this will only

delete files &

subfolders inside

it, it does not

delete `project` folder.

safer version for `rm -r` is

3) `rm -ri` → interactive (ask before deleting each file or folder)
↓ recursive
remove

4) `rm filename` → deletes only files, not folders.
↓

(name of the file you want to delete)

`rm cloud/*`

↓ this will delete all files inside `cloud` folder but not subfolder.

safer version to delete file

5) `rm -i filename` → It will ask yes (y) or no (n) before deleting it.

6) `rmdir` → It is used to delete an empty folder only.

e.g. `rmdir myfolder` → it will delete a folder named `myfolder` if it is empty.

if it contains any files or subfolders it will show an error.

7) `rm -rf` → used to force delete a folder and everything inside it, without asking.

↓
force
↙
(don't ask even if files are protected)

* NOTE : This command can delete your whole system
`rm -rf /`

* In very rare cases recovery is possible if we ~~you~~ run commands like `rm -r` or `rm -rf`

You use special file recovery tools like
testdisk, extundelete, photorec (linux)

it works only if
→ you act immediately
→ data hasn't been overwritten.

Even then success is not guaranteed.

8) `cat` → stands for concatenate.
mostly used to view contents of a file.

`cat notes.txt`

`cat notes1.txt notes2.txt` → view multiple files.

9) `echo` → is used to print text or variable in the terminal.

`echo "hello alok"` → this will print hello alok
rewrites new text every time.

`echo "hello alok" > domofile.txt` → (this is existing file)
this will add text to
existing.

`echo "hello abhay" >> domofile.txt` → append text to a file.

`echo "hello world" > myfile.txt` → this will create a file and add the text into it.
↙
there is no such file created

10) `zcat` → used to view contents without extracting a file.

it will work with `.gz` file only.

11) `head` → shows first lines of file
(by default it shows first ~~10~~ 10 lines)

If we write

`head file.txt` → it will show first 10 lines

`head -n 5 file.txt` → first 5 lines will be displayed.

12) `tail` → shows last lines of a files
(it shows 10 lines by default)

`tail file.txt` → last 10 lines.

`tail -n 5 file.txt` → shows last ~~10~~ 5 lines of a file.

13) `tail -f` → used to watch the end of a file in real time as it gets updated.

mostly used to monitor log files.

`tail -f log.txt`

→ To stop it press `Ctrl + C` → to exit line view.

14) `less & more` → used to view large text files page by page in the terminal.

`more file.txt` : show one screen of text at a time.
Press Enter to scroll line by line
Press space to scroll page by page
Press q to quit.

* cannot scroll backward.

less: more advanced & flexible.

scroll forward with space, backward with b
use arrow key $\downarrow \uparrow$ for line by line navigation.
press q to quit.

Search text:

/text $\rightarrow$ search forward for "text"

?text $\rightarrow$ search backward for "text"

n $\rightarrow$ next search match.

N $\rightarrow$ prev search match.

eg: less filename.txt

$\downarrow$ or Enter $\rightarrow$ scroll down one line.

$\uparrow$ $\rightarrow$ scroll up one line

space $\rightarrow$ scroll down one page

b $\rightarrow$ scroll up one page

15) cp $\rightarrow$ copy : It is used to copy files or folders from one place to another in linux.

Basic Syntax: cp source destination

cp file.txt denops/

cp file1.txt file2.txt denops $\rightarrow$ copies multiple file to a folder.

- rm file1.txt file2.txt file3.txt $\rightarrow$ this will delete multiple files at a time.

* cp file.txt backup.txt

$\downarrow$
this means

= copy the file named file.txt and create a new file backup.txt with the same content.

(if not existed)

= if there is a file already named with backup.txt.
this command will overwrite with the content of file.txt
we can use cp -f file.txt backup.txt to prevent accidental overwrite.

`cp -r` → -r means recursive - copy everything inside the folder (including subfolders and files)

`cp -r folder1/ folder2/`

this copies folder 1 into folder 2

if folder 2 does not exist, it will be created.

It is quick and silent → will overwrite files in the destination without warning.

`cp -riv folder1/ folder2/`

Diagram showing the breakdown of `-riv`:

- `-r`: recursive (copy folders)
- `-i`: interactive (asks before overwriting)
- `-v`: verbose (shows what's being copied)

Let suppose there are 2 folders cloud & denops and denops contains files → `name.txt`, `newfile.txt`

If we want to copy the file `newfile.txt` from denops folder to cloud folder

→ `cp denops/newfile.txt cloud/`

we can also copy multiple files from denops folder to cloud

→ `cp denops/newfile.txt denops/name.txt cloud/`

`rm *` → this will delete all files inside a folder if we are already inside the folder

If we want to copy whole cloud folder to denops folder

→ `cp -r cloud/ denops/`

16) **mv (move)** → used to move files or folders to another location.
→ rename files or folders.

take an example

suppose we have 2 folder and 3 files

cloud demofile.txt desktop myfile.txt newfile.txt

mv demofile.txt cloud/ → this will move demofile.txt inside cloud folder

if in cloud folder demofile.txt already existing, by default the existing file will be overwritten without warning.

To avoid this use

mv -i demofile.txt cloud/ → this will ask before overwrite.

if desktop folder contain two files

new-desktop-file.txt name.txt

we want to ~~copy~~ move name.txt file to cloud folder

mv name.txt ../cloud/

- **mv oldname.txt newname.txt** → This renames the file
It moves oldname.txt to newname.txt & original file is gone.

cp oldname.txt newname.txt → It creates a new file called newname.txt with same content as oldname.txt.

`mv file1.txt file2.txt cloud1` → moves multiple files.

`mv cloud1 devops1` → if existing, moves it & if not ~~creates devops folder~~ ~~and then moves it~~.
cloud will be renamed to devops.

➤ `mv devops1 linux-for-devops` → this will rename the folder named devops to linux-for-devops

If we want to move all files inside devops folder to cloud folder

`mv devops/* cloud1` → only moves files, not hidden files.
↓
this means all files & subfolders inside devops.

12) `wc` (word count) → It is used to count ~~the~~ lines, words and characters in a file.

~~the~~ Suppose myfile.txt contains
Hello world
this is a text

`wc myfile.txt` → output → 2 6 27 myfile.txt
↓ ↓ ↓
no. of words characters
lines (including spaces & newlines).

`wc -l myfile.txt` → line count only

`wc -w myfile.txt` → word count only.

`wc -c myfile.txt` → character count (bytes)

➤ `wc myfile1.txt myfile2.txt myfile3.txt`

Imp

Soft link (Symbolic link)

a soft link is like a shortcut to a file.

You create a soft link that points to another filename.

If original file is deleted, the link breaks.

Steps

a) Create a file

```
echo "this is the original file" > original.txt
```

```
cat original.txt >> this is the original file.
```

b) Create a soft link

```
ln -s original.txt link.txt
```

```
cat link.txt >> this is the original file.
```

c) Edit the original file.

```
echo "file was changed" > original.txt
```

```
cat link.txt >> file was changed.
```

d) `rm original.txt`

```
cat link.txt
```

The link is now broken, because it was only pointing to the file by name and the original is gone.

It is useful to point to files in other folders

Hardlink :-

A hardlink is link giving another name to some file.

Imagine you have a file called original.txt

→ Create a file

```
echo "this a original file" > original.txt  
cat original.txt
```

13) `ln original.txt link.txt`

Now you have two names for the same file
content:

```
original.txt  
link.txt
```

```
cat link.txt    >> this a original file
```

Both files show same data

→ Change content using hard link.

```
echo "update through hard link" > link.txt
```

Now check both files

```
cat original.txt    > >> update through hard  
cat link.txt        link.
```

delete original file

```
rm original.txt  
cat link.txt
```

>> update through hard link.

It still works, because the file content still exists.
You only removed ~~one~~ name.

→ Check inodes (to prove they are same file).

`ls -li`

```
123456 - - - - - link.txt  
123456 - - - - - original.txt  
└── some inode no, same file content.
```

20) cut → used to cut out and display specific parts of each line from a file or output.

cat name.txt → hello world
this is a test.

cut -b 1 name.txt >> H
↓
byte T

cut -b 1-5 name.txt >> Hello
This-

cut -c 11 11 >> Hello
↓
character this-

cut fields by delimiters (comma separated value)

echo "name,age,city" > info.txt

cut -d',' -f2 info.txt >> age

cut from a file students.txt

↳ John, 21, Delhi
Dara, 22, Mumbai

cut -d',' -f1 students.txt >> John
Dara

-b, -c → select characters by position, b is for bytes.

-d → set delimiter (like comma, colon, space)

-f → select field / column numbers.

21) tee → it displays the output on the terminal (screen)
→ and saves the same output into a file.

eg: echo "Hello Linux" | tee hello.txt

it will show output on the screen & as well as
create a hello.txt file and save this into the file.

>> Hello Linux

cat hello.txt

>> Hello Linux.

echo "Another line" | tee -a hello.txt



>> Another line
cat hello.txt

this means append
instead of replacing
the file content.

>> Hello line
>> Another line.

22) sort → used to sort lines of text files in
alphabetical or numerical order.

> a name.txt file contains names

Alok
Dev
Piyush
Shivam
Bhupendra

→ sort name.txt

Alok
Bhupendra
Dev
Piyush
Shivam

∴ sorts by
Capital to
small

> a number.txt file contain

1
5
2
20
9

→ sort number.txt we have to use

1
5
2
20
9

→ sort -n number.txt

↓
sort numerically
not alphabetically

1
2
5
9
20

sort -r name.txt

↓
sort in reverse
order

>> Shivam
Piyush
Dev
Bhupendra
Alok

sort -u names.txt

↓
unique (remove duplicate
lines)

> sort by specific column (field)

data.txt

Alice 23
Bob 18
Charlie 30

sort -k2 -n data.txt

↓
sort by
sec. field ↓
numerical
sort

>> Bob 18
Alice 23
Charlie 30

23) diff → diff command is used to compare two text files line by line.
It shows what is diff. b/w them.

diff file1.txt file2.txt

diff -y file1.txt file2.txt

↳ side by side comparison.

diff -i file1.txt file2.txt

↳ ignore case differences

diff -u file1.txt file2.txt

↳ ignore whitespace changes

✓ useful for checking two versions of code or text

Symbols in diff output

< line from first file
> line from second file
a add
d delete
c change

in output if we see
1, 7c1, 3

line 1 to 7 in file 1
have been changed to
line 1 to 3 in file 2.

vi and vim

are text editors used in terminal, they let you view,
edit and write files from the command line

vim name.txt → it will open vim editor and
show file content.

Now press i → it will open insert mode.

then insert anything, update or delete then

press esc to come out from insert mode

then type :wq to write and quit.

Now if you type cat name.txt → all the edits
will be showing.

24) `df` → used to display info about disk space usage or file systems.

`df` → shows disk space usage for all mounted file systems.

`df -h` → shows disk space in human readable format (like MB, GB)

`df -T` → displays file system type. (ext4, btrfs etc)

`df -a` → includes dummy file systems (like proc, sysfs)

`df -h /home` → shows disk usage of the /home directory specifically.

`ls -a` → used to show hidden files.

Commands for managing & viewing processes:-

1) `ps` → shows currently running processes.

`ps -e` or `ps -A` → lists all running processes.

`ps aux` → detailed info on all processes.

2) `top` → displays real time list of processes.
use `q` to quit.

sort by CPU usage (default) or press `m` for memory

`htop` → an enhanced, colorful and interactive versions of `top`.

3) `kill` → used to stop (terminate) running process using its process ID (PID)

`kill 1234` or `kill -15 1234` are some.
PID

↘ asks gracefully the process to stop.

kill -9 1234 → forcefully kills the process immediately

pkill → kills processes by name.

How can we delete specific line inside a file:

sed -i 4d file.txt
↓
delete line no. 4

if we used sed 4d file.txt

This will just display the file content without line 4

If we want to delete the whole content of a file without deleting the file:-

: > filename.txt
↳ name of the file

if we want to delete range of lines like (5-8 lines)

sed -i 5,8d file.txt
↳ this will delete lines from line no 5 to line no 8.

sed -i \$!d file.txt → delete all blank or empty lines.

if we want to delete specific no. of line

sed -i -e 2d -e 4d -e 6d file.txt

this will delete lines 2, 4, 6 from file.txt.

Note → always write highest line number last when using -e.

Users and file management in Linux

System level commands:-

1) `uname` → used to show system info.

`uname` → shows system name like linux in ubuntu, centos etc and darwin in macos.

`uname -a` → shows all info (kernel, hostname etc)

2) `uptime` → shows how long the system has been running.

3) `date` → displays current date and time.

4) `who` → shows ^{all} users currently logged in to system.

5) `whoami` → prints currently logged in users username. (your username)

6) `which` → which command shows the path of the executable file that ~~will~~ would run when you type a command in terminal.

e.g. `which python` (∴ used to know where a command/program is installed)
`>> /usr/bin/python`

this means when ~~you~~ you type python, the system is using the executable located at `/usr/bin/python`.

7) `type` → tells what kind of command it is:

Built-in, alias, function, external command

`type ls`, `type cd` etc

8) `id` → used to display user ID (UID), group ID (GID) and the groups the current or specified user belongs to

eg: `id`

`uid = 1000(alok) gid = 1000(alok) groups = 1000(alok), 27(sudo)`

`uid = 1000(alok)` → your user id is 1000, username alok.

`gid = 1000(alok)` → your primary gid is 1000, group name alok.

`groups = 1000(alok), 27(sudo)` → You are part of alok & sudo groups.

9) `sudo` → it stands for superuser do.

`sudo` lets a normal user do admin/root level tasks.

10) `shutdown` → used to turn off, restart or schedule a shutdown of your system safely.

`sudo shutdown` → this shuts down the computer after 1 min.

`sudo shutdown now` → immediate turns off the system.

`sudo shutdown +10` → shuts down after 10 min.

`sudo shutdown -r now` → restarts the system now.

`sudo shutdown -c` → cancels a scheduled shutdown.

schedule shutdown at specific time :-

`sudo shutdown hh:mm` → time in 24 hour format

`sudo shutdown 22:00` → this will shut down system at 10:00 PM.

`sudo shutdown 22:00` "system will shutdown at 10 PM for updates"

if we use 01:00 → 1 AM
13:00 → 1 PM
06:45 → AM
18:45 → PM
12:00 → 12 PM (noon)
00:00 → 12 AM (midnight)

11) reboot → used to restart the system.

`sudo reboot` → immediately restarts your computer.

`sudo shutdown -r now` → restart system right now.

`sudo shutdown -r +10` → restart system in 10 min.

`sudo shutdown -r 23:00` → reboot at 11:00 PM.

12) apt → application package manager or software manager for your terminal.

a tool used on debian-based systems (like ubuntu) to install, update or remove software from the system.

Always run

`sudo apt-get update`

before installing or upgrading anything so

your system uses the latest versions.

`sudo apt-get update`
package manager for
debian / ubuntu systems.

It does not

- install any new software
- upgrade any packages.
- change anything except the package list.

`sudo apt-get update` does not install or upgrade anything, it just refreshes the list of available software from all the repositories.

if we want to install anything from internet use
`get`

`sudo apt update` → checks for new version of packages.

`sudo apt upgrade` → installs the newest versions.

`sudo apt install <name>` → install a program.

`sudo apt remove <name>` → uninstalls the program
(keeps config files)

`sudo apt purge <name>` → ~~uninstalls~~ uninstalls +
deletes config files.

`sudo apt autoremove` → removes unused packages
automatically.

Always run `sudo apt update`, before installing software.

`sudo apt clean` → deletes all downloaded package
files (in cache)

`sudo apt autoclean` → deletes only outdated
cached packages.

`sudo apt search <keyword>` → searches for packages.

`sudo apt show <package>` → shows detailed info about package.

`sudo apt list --installed` → lists all installed packages.

`sudo apt list --upgradable` → shows package that can be upgraded.

`sudo apt policy <package>` → shows which version will be installed & from where.

you can also install multiple packages

✓ `sudo apt install git curl vlc`

13) `yum` → (yellowdog updater modifier)
used in : old versions of Red Hat, CentOS,
Fedora.

RPM based package manager.

↓

(Red Hat package manager)

`sudo yum update`

`sudo yum install nginx`

`sudo yum remove nginx`

14) `dnf` (dandified yum) → used in newer versions of Fedora, CentOS, RHEL

replaces yum because it's faster and better)

same syntax as yum but better performance,
better dependency handling.
RPM based package manager.

15) pacman → Arch Linux and Arch-based distros
(eg. manjaro)

uses .pkg, .tar, .zip packages

16) portage → used in Gentoo Linux, Funtoo Linux,
ChromEOS.

source based package manager (compile from
source)

emerge firefox → install firefox from source

emerge --unmerge firefox → remove firefox.

Users & group management commands:-

1) useradd → create a new user acc. on the system.

sudo useradd -m alok → this will create the user
in home directory.

2) passwd → used to set or change a users password

→ to set password for → sudo passwd alok
username alok

→ type only passwd to change your own password

→ to change or update another users password

sudo passwd alok
→ username.

To force password change at Next login

sudo chage -d 0 username

`sudo chage -l username` → shows password expiration info

`sudo passwd -l username` → locks a users acc.
(disable login)

`sudo passwd -u username` → unlocks a users acc.

`sudo passwd -e username` → expire a user password immediately.

3) `su` → used to switch users acc. , most commonly to switch to root user.

`su alok` → this will ask you to type password, then login to alok user from primary user.

type exit to go back to primary user

if we type only `su` with out any username, it tries to switch to the root user by default.

if the root is enabled → you have to enter root users password.

if the root is disabled → you will get an error.

for ubuntu based systems.

if root is disabled and you want to switch to root, you use

`sudo su`

sudo su basically runs the su command as root.
→ you become root temporarily

To truly enable the root acc. on ubuntu

set password for root

sudo passwd root

sudo passwd -l root → disables the root

to exit from root user

type exit or press ctrl + D

if we use sudo su username it switches to another user.

→ cat /etc/passwd → displays all user acc. info stored on the system.

Format is

username : x : UID : GID : comment : home-directory : shell

alok : x : 1001 : 1001 : Alok Patel... : /home/alok : /bin/sh
↓ ↓ ↓ ↓ ↓
username UID GID comment/ Home default
fullname directory shell

password
placeholder
(always x)

↓
actual password
is in /etc/shadow

↓
you can add comment to username by
sudo useradd -m -c "Alok Patel"

→ if you want to update
comment for user su
sudo usermod -c "Alok Patel"
alok.

if there is no comment, it
will add it to username.

4) `userdel` → to delete a user whether it is newly created or existing.

`sudo userdel ok` → this will remove the user from system, but does not del. their home directory or files.

`sudo userdel -r username` → this will delete user and all existing files and home directory (`/home/username`)

if the user is logged in, the command may fail.

we can force delete a logged in user

✓ `sudo pkill -u username`
✓ `sudo userdel -r username`

5) `groupadd` → lets you create a new group that users can belong to.

`sudo groupadd groupname`

`sudo groupadd developers`

↳ this creates a group called developers.

we can also check whether group is added

`cat /etc/group` or `getent groups groupname`
↓
group name.

if we want to give a group Id

`sudo groupadd -g 2001 testers` → creates a group testers with GID 2001

* If me type only

`sudo useradd alok` → this will create a user but not in home directory.

```
sudo useradd jithalal
sudo useradd iyer
sudo useradd tappu
sudo useradd bhide
sudo groupadd denops
sudo groupadd tester
```

Add guthalal word identifier to devops group

sudo gpasswd -a jithalal devops
" " " subunter devops.

To check this

getent group dumps.

we can also add multiple ^{users} to a group at the same time

suda gpassund -m iyer, tappu, bhide tester

this will add these three users to tester group.

5) groupdel $\rightarrow$ delete group

side group del tester

- ↳ deletes the tester group.

How can we remove user from group

`sudo gpasswd -d jethalal demops`



this removes jethalal from demops group

2nd method

`sudo deluser jethalal demops`



removes jethalal from demops group.

File permission commands

1) `ls -l` → used to list files and directories in long format.

`ls -l`

» `drwxrwxr-x` - - - something
`-rwx-rw-r--` - - - something

user group other user
`drwxrwxr-x`

these are permissions

this 'd' identifies whether it is directory or not, if it is directory, it starts with 'd'.
read write execute

→ if we want to change permissions for other users.

0	-	-	-
1	-	-	X
2	-	w	-
3	-	w	X
4	r	-	-
5	r	-	X
6	r	w	-
7	r	w	X

`drwxrwxr-x`
7 7 5

» `chmod 777 cloud` → directory name.

this will change the permission → `drwxrwxrwx`

ls -l gives a detailed view of
 file permissions
 ownership
 size
 modification time
 file name.

» drwxrwxrwx 2 ubuntu ubuntu 4096 Jul 8 12:19 cloud

↓ ↓ ↓ ↓ ↓

file permissions no. of links owner name group name file size in bytes last modified date & time file or dir name

ls -lh → human readable
 ↓
 if we use this we can easily understand file sizes in KB, MB etc)

ls -la → list all files including hidden ones.

2) umask → this command sets default permissions values for new files and directories that you or processes create.

Default permission before umask.

files → 666 rw-rw-rw-
 directories → 777 rwxrwxrwx

umask → user file-creation mode mask (4 digit octal no.)

How umask work:-

Let's say your umask is 022 for a file

default 666

umask 022

final 644 → rw-r--r--

3) `chown` → this is for changing ownership.
It's used to change the owner and
or group of files or directories.

```
sudo chown alok demofile.txt
```

↳ this will change the
owner to alok from
ubuntu.

if we want to change owner & group both

```
sudo chown ubuntu:devops demofile.txt
```

sets owner to ubuntu & group to devops.

to change only group

```
sudo chown :devops demofile.txt
```

↳ changes just the group to devops.

```
sudo chgrp devops demofile.txt
```

Compression command:-

`chmod` → this command is used to change file
permissions.

r → 4

w → 2

x → 1

1) Zip → used to compress files and folders into a
.zip archives.

e.g. - `zip myfile.txt myfile.zip file1.txt file2.txt`
↳ name of the zip file we can choose by ourselves
these are files we want to zip.

`zip myfile.zip file1.txt file2.txt`

This command creates a `myfile.zip` file containing `file1.txt` and `file2.txt`.

zip a folder

`zip -r myfolder.zip myfolder/`

↳ folder name

adding a file to an existing zip archive:-

`zip yd.zip newfile.txt`

↓
existing
zip

↓
the file we want to add to an
existing zip.

zip with password protection:-

`zip -e secure.zip file1.txt` → this will encrypt only single file.

⇒ To encrypt multiple files

`zip -e secure.zip file1.txt file2.txt file.pdf`

⇒ To encrypt a folder and its contents

`zip -er folder.zip myfolder/`

⇒ If we want to extract to a specific folder:

`unzip secure.zip -d newfolder/`

This will unzip `secure.zip` into the folder named `newfolder/`

Exclude specific files or folders:-

```
zip -r myproject.zip myproject -x "*.log" "temp/*"
```

"*.log" → includes all .log files

"temp/*" → includes everything inside temp folder.

If we want to delete any file, folder or zip file.

For file:-

rm file.txt → deletes a single file

rm file1.txt file2.txt file3.txt → this will delete multiple files in one go.

rm *.txt → deletes all .txt files.

rm *.zip → deletes all .zip files inside the current directory.

rm -f file1.txt file2.zip → force delete

for folder

rm -r foldername

rm -rf foldername → deletes everything inside a folder forcefully.

for zip files

rm file1.zip secure.zip file2.zip.

If we want to delete all zip files in a specific folder.

rm |path|to|folder|*.zip

2) gzip → compress files.

gzip file.txt

this compresses file.txt and creates file.txt.gz
the original file.txt is deleted (by default)

gzip file.txt name.txt → compress multiple files.

gzip -k file.txt → keeps original file while compressing.

3) gunzip → used to decompress .gz files.

gunzip file.txt.gz

decompresses file.txt.gz back to file.txt
the .gz file is deleted after extraction.

if we want to keep the .gz file.

gunzip -k file.txt.gz

gunzip -k file1.txt.gz file2.txt.gz file3.txt.gz

↳ decompresses multiple .gz files

creation archive → filename of archive

↑

tar -cf archive.tar file1.txt file2.txt

4) tar → tape archive

untar → tar -xvf archive.tar

↓
extract → filename of archive.

To create a compressed tar archive (tar.gz)

tar -cvzf archive.tar.gz file1 file2 folder/

↳ compresses with gzip

To extract

tar -xvf archive.tar.gz

NETWORKING COMMANDS:-

1) ping → used to test the connection b/w your computer and another computer (or server) over a n/w.

when you type ping, your computer sends ICMP echo request packets to a target (like a website) and waits for ICMP echo reply packets.

→ ping runs forever by default.

→ Press Ctrl + C to stop it

→ it is used to check if other device is reachable.

→ measure n/w speed and stability. (latency)

→ Diagnose internet or n/w issues.

2) Netstat → this command is used to monitor & display n/w related info. such as

- Active connections
- open ports
- Routing tables
- listening services

It helps you diagnose n/w issue or check what services are running or listening on certain ports.

Imp.

To use netstat & various other commands of networking firstly install net-tools

`sudo apt install net-tools`

`netstat -a` → shows all n/w connections

`-l` → shows listening ports

`-t` → TCP connections only

`-u` → UDP connections only

`-p` → shows process ID (PID) & program name

`-n` → shows numeric IPs and ports.

Displays IP addr. & ports no.

`-r` → routing table (kernel routing table)

`-i` → interface stats.

Displays data like packets sent/received per n/w interface

On modern systems, ss is faster and more powerful.

`ss -tulnp`

2) `ifconfig` → used to view and configure
n/w interfaces like
Ethernet (`eth0`)
Wifi (`wlan0`)
loopback (`lo`)

* `ifconfig` is deprecated on newer linux systems,
replaced by `ip` command (`ip addr, ip link`)

Key info which it shows.

`eth0` → interface name

`inet` → IPv4 addr

`inet6` → IPv6 addr

`netmask` → subnet mask

`broadcast` → broadcast addr.

`ether` → MAC addr.

`RX/TX` → packets → Data received/sent.

`sudo ifconfig eth0 192.168.1.100` → assign an IP
addr. to an
interface.

`sudo ifconfig eth0 up` → enable on interface

`sudo ifconfig eth0 down` → disable on interface.

`sudo ifconfig eth0 hw ether 00:11:22:33:44:55`
↓
change MAC addr temporarily.

4) Traceroute / tracpath → both commands shows the path your data takes from your computer to destination (like google.com)

To use traceroute / tracpath we need to install

sudo apt install traceroute

→ traceroute google.com

→ tracpath google.com

* use traceroute if you want more control, detailed output or need to use diff. protocols.

use tracpath if you want ~~to~~ a quick and simple trace without needing sudo.

5) mtr → stands for my traceroute.

It gives a realtime, detailed view of the route packets take to reach a destination & how each hop behaves over time

its like traceroute + ping

traceroute → one time path tracing.

mtr → live detailed, realtime network diagnostics.

sudo apt install mtr

mtr -r -c 10 google.com → runs 10 test packets, prints a summary then exits.

↓ ↓
report count
mode (limit the no. of
 pings, used with
 -r) on mtr

6) nslookup → used to query DNS to get
↓ ↓ info. about domain names & IP
name server addr.

Think of it as "What's the IP of this website?"

nslookup google.com

It shows

Server → DNS server used to answer your query.

Name → the domain name you looked up

Address → IP address of that domain.

→ You can also find out domain name for an IP address.

nslookup 8.8.8.8 → this is called reverse dns lookup.

7) telnet → telnet is mostly used for testing connectivity to specific ports.

telnet google.com 80

↓
this tries to connect google.com on port 80 (HTTP)

If the connection works you will see a blank screen or some server response. If it fails, you'll get a connection error.

It is used for:

- checking if a port is open on a server.
- testing services like HTTP, SMTP, FTP
- Diagnosing firewalls & n/w issues.

previously telnet was used for remote login
but now ssh is used because it is more safe & secure.

eg: testing an email server

telnet smtp.gmail.com 587

- 8) hostname → used to
- view the name of your computer (host)
 - temporarily or permanently set the host name.

Its like your computers "name on the network".

just type hostname

sudo hostname new-name → change the hostname until next reboot.
after reboot it goes back to old name.

sudo hostnamectl set-hostname new-hostname

↓
used on modern linux system and is recommended and permanent way (works across reboots)

hostname -I → show IP addresses.

hostnamectl → show & set permanent hostname.

hostname → show current hostname.

- 9) ip → ip command is modern & powerful replacement for older tools like ifconfig, route & netstat.

ip addr show → show all ip addresses.

or

ip -a

1) `iwconfig` → wireless configuration tools.
this command is used to view and config
wireless n/w interfaces (like wlan0) in linux.
like `wlan0`

Firstly install wireless-tools

`sudo apt install wireless-tools`

1) `dig` → powerful DNS lookup tool used to
query DNS records and troubleshoot
DNS problems.

It gives detailed, flexible output -

→ Find IP addresses of a domain.
look up the name servers (NS)
Reverse DNS (IP to domain)

`dig google.com`

2) `whois` → `whois` command is used to lookup info.
about domain name or IP address.

like → who owns it, when it was registered,
when it expires.

info → who owns the website.

`whois google.com`

Firstly install `sudo apt install whois`

3) arp :- command which maps IP addresses to MAC addr.
on a local n/w.

It is used only within the LAN.

most modern systems now prefer :
ip neigh.

4) ifplugstatus :- used to check whether a n/w cable
is plugged in or not for a given
n/w interface

`sudo apt install ifplugd`

Network Interface :

It is the connection point b/w your computer and a
n/w.

Think of it as the doorway your computer uses to send
& receive data over n/w.

Types → Ethernet → eth0, enp3s0

Wifi → wlan0, wlp2s0

loopback → lo

Virtual → created by software
(e.g. docker, VPNs)

tlm, dockero.
↓
VPN
tunnel

→ curl → used to make ^{n/w} requests like downloading
files, testing APIs or submitting forms.

client orl

used to

→ fetch web pages or data

→ ~~Test~~ REST APIs (GET, PUT, POST, DELETE)

→ upload/download files.

→ communicate over many protocols like
KTTD, HTTPS, SMTP, FTP etc.

1) fetch a web page

`curl https://google.com`

↳ displays HTML content of the website in the terminal.

2) Download a file.

`curl -O https://example.com/file.txt`

↳ saves the file with its original name.

3) `curl -X POST`
`curl -X GET`

16) `wget` → used to download files

→ download files from HTTP, HTTPS or FTP servers.

→ Resume broken downloads.

→ download entire website recursively.

3) download a single file

`wget https://example.com/file.txt`

`wget -O myfile.txt https://example.com/file.txt` → save with diff. file name.

2) resume a broken download:

`wget -c https://abcd.com/bigfile.zip`

↳ continue on incomplete download.

`wget -r https://abcd.com/` → download a website recursively

3) download & save to specific ~~path~~ directory:

`wget -P /home/user/Downloads https://abcd.com/file.txt`

jq → when you work with APIs or config files we get data in JSON format.

jq makes JSON format data to pretty print (organizes data so that we can understand it easily)

combine with API

```
curl -X GET https://api.abcd.com | jq
```

→ jq is also used to extract specific fields.

→ filters based on values.

→ modify JSON content.

17) watch → used to run another command periodically showing the output in real time.

→ automatically repeats a command every 2 sec (by default)

→ updates the terminal with fresh output.

→ helpful for live monitoring.

◀ monitor disk usage.

```
watch df -h
```

```
watch -n 5 free -h
```

→ set interval to 5 sec.

press ctrl + C to stop.

18) nmap → network mapper.

powerful CLI tool used to scan n/w, discover hosts, open ports, services and OS

→ Finds live systems on a n/w.

→ Find open ports on a system

→ guess OS

→ detect vulnerabilities.

Linux volume management

lsblk: command in linux used to list info. about all available block devices.

(like disks, partitions & lvm volumes)

xvda → root volume

↓

/dev/xvda

↓
(device)

df -h
↳ human readable

xvda (root volume) becomes /dev/xvda when it is mounted to device. (1)

useful option :-

lsblk -f → show filesystem type, label, mount point

lsblk -p → show full device path.

lsblk -o NAME, SIZE, TYPE, MOUNTPOINT → show custom columns.

lsblk -a → show all devices including empty ones.

lsblk -d → show only disks, not partition.

→ df -h / → shows which device mounted on / (root)

while choosing device name in creating volume

sda, sdb ---- sde a to e → disk device names.

sda → first disk, sdb → second disk like this

└ sda1 → first partition (might be root /)

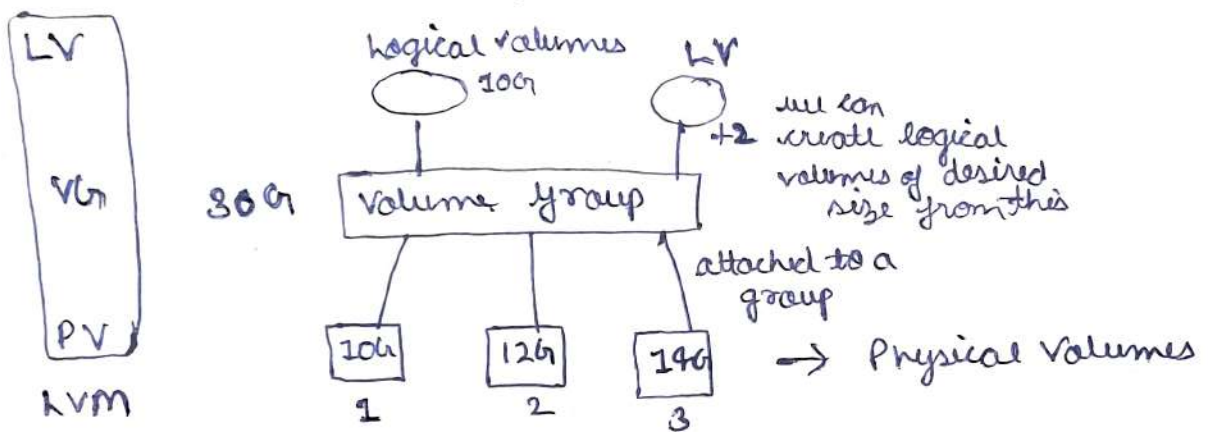
└ sda2 → second partition (might be swap)

sdg --- ~~sdg~~ sdp → data volumes.

Logical Volume Manager (LVM):-

→ It's a flexible way of managing disks and partitions in Linux.

→ Instead of using fixed partitions (/dev/sda1, /dev/sda2, etc). LVM lets you create logical volumes that can be resized, extended or moved easily.



Key components

1) Physical Volume (PV) :- actual disk or partition.
e.g. /dev/sda2, /dev/sdb

2) Volume Group (VG) : contain multiple disks combined.
e.g. vgo

3) Logical Volume : partitions created from the VG, mounted like normal filesystem
e.g. /dev/vgo/root, /dev/vgo/home

Suppose you have 2 disks

/dev/sda2 (50GB)

/dev/sdb (100GB)

} these are blocks, disks, or volumes.

→ Convert them into physical volumes (PV)

→ Combine them into one volume group (VG) → 150GB

→ Create logical volumes (LV):

- /dev/vg0/root → 40GB

- /dev/vg0/home → 80GB

- /dev/vg0/var → 30GB

Now if /home fills up you can resize it without partitioning.

LVM commands:-

`pvs` → show physical volumes

`vgs` → show volume groups

`lvs` → show logical volumes.

→ create physical volume (PV)

`pvcreate /dev/xvdf /dev/xvdg /dev/xvdh`

→ this converts disks to physical volumes.

→ create volume group (VG):-

`vgcreate` `twsg` `/dev/xvdf` `/dev/xvdg`

↓
volume group
name

disks we want to
combine.

→ create logical volume:

lvcreate -L 10G -n tms-lv ~~vg-data~~ tms-vg

↓ ↓ ↓ ↘ from which
size of name name for volume group
volume logical
we want volume
to create

→ this will create a 10GB logical volume name tms-lv inside tms-vg

→ pvdisplay → this command shows detailed info. about physical volumes (PVs) in LVM

vgdisplay → shows info. about volume group.

lvdisplay → display info. about logical volumes.

≡ Diff b/w volume attach & mount:

volume attach means adding block to a disk
and mount. means making it usable.

Mounting:-

In Linux, there are no drives like C, D in windows. Instead of, everything is a part of single directory tree starting from /.

→ mounting means attaching a storage device (disk, partition) to a directory, so you can access its files.

→ unmounting means detaching the device from the directory.

e.g: mounting: firstly be a root user `sudo su`
`mkdir /mnt/tus-lv-mount`

→ After this we have to format logical volume to prepare a disk / partition with a file system, so the OS can store and manage files on it.

In linux formatting command is

`mkfs.ext4 /dev/tus-vg/tus-lv`
↓
it is a filesystem
path to format

→ `mount /dev/tus-vg/tus-lv /mnt/tus-lv-mount`
↓
logical volume (path) source
↑
mounted logical volume to this location. destination

For unmounting:

`umount /dev/tus-vg/tus-lv`

Extending a logical volume:-

`lvextend -L +10G /dev/tus-vg/tus-lv`
`resize2fs /dev/tus-vg/tus-lv` # resizes filesystem

Reducing a logical volume:-

`lvreduce -L -5G /dev/tus-vg/tus-lv`
`resize2fs /dev/tus-vg/tus-lv`

→ Remove logical Volume:-

umount /mnt/tus-lv mount
lvremove /dev/tus-vg/tus-lv

→ Remove Volume group:-

vremove tus-vg

→ Remove Physical volume:-

pvremove /dev/xvdy /dev/xvdg /dev/xvdh

prcreate → vgcreate → lvcreate → mkdir → mkfs → mount

lvextend / lvreduce → resizing.

lvremove / vremove / prremove → cleanup

Mounting disk volume:

→ mkdir /mnt/tus-disk-mount

→ format the disk

↳ mkfs -t ext4 /dev/xvdh

mount /dev/xvdh /mnt/tus-disk-mount